

Session 2

TypeScript & State



Quick Recap

- Node & npm
- package.json
- First React component

TypeScript vs JavaScript

What is TypeScript?

TypeScript adds syntactic sugar on top of JavaScript, allowing developers to add types

Question:

Why would we want to do this?



Some general JavaScript issues

- Runtime errors
- Hard to refactor
- No type safety
- A few more



TypeScript tries to solve this

- Helps catch errors earlier with types
- Improves readability
- Can be seen to improve developer experience

JavaScript is valid TypeScript

TypeScript is NOT valid JavaScript



Adding Types to Variables

```
let name: string = "Jason";  
let count: number = 10;  
let isActive: boolean = true;
```

👉 Types are optional, but useful

Type Inference

```
let name = "Jason";  
name = 123; // ❌ Error
```

👉 TypeScript infers the type automatically



Arrays

```
let numbers: number[] = [1, 2, 3];
```

```
let names: string[] = ["Ash", "Misty"];
```



Object Types

```
type User = {  
  name: string;  
  age: number;  
};
```



Optional Properties

```
type User = {  
  name: string;  
  age?: number;  
};
```



`age` is optional



Union types

```
type Name = "Ash" | "Misty" | "Brock";
```

```
let trainerName: Name = "Ash"; // gives intellisense  
trainerName = "Jason"; // ❌ Error
```

```
let myVar: Name | number | User;
```

```
myVar = "Brock";
```

```
myVar = 123;
```

```
myVar = { name: "Jason", age: 30 };
```

```
myVar = true; // ❌ Error
```



Nullish Coalescing

```
let name: string | null = null;  
  
let displayName = name ?? "Default Name"; // Output: Default Name
```

👉 Only falls back if null or undefined



Discriminated Unions

```
type Status =  
  | { type: "loading" }  
  | { type: "success"; data: string }  
  | { type: "error"; message: string };  
  
let stat: Status = { type: "loading" };  
stat = { type: "success", data: "It was super effective" };  
stat = { type: "error", message: "Magikarp used Splash... but nothing happened" };
```

👉 Useful for handling different states



Multi-Paradigm Language

```
interface IPokemon {  
    name: string;  
    shoutMyName(): string;  
}  
  
class Pokemon implements IPokemon {  
    name: string;  
  
    constructor(name: string) {  
        this.name = name;  
    }  
  
    shoutMyName() {  
        return this.name.toUpperCase();  
    }  
}
```

Just because it walks like a duck and talks like a duck
doesn't mean it's entirely a duck...
it also has features of other animals

While this is possible, as in consulting, "it depends" on the use-case.



Functional Alternative

React now mostly focuses on functional

```
type Pokemon = {  
  name: string;  
  shoutMyName: () => string;  
};  
  
function createPokemon(name: string): Pokemon {  
  return {  
    name,  
    shoutMyName: () => name.toUpperCase()  
  };  
}
```



Let's apply TypeScript

```
function DevCampButton({ origin, children }) {  
  return (  
    <button onClick={() => alert(`Hello from ${origin}`)}>  
      {children}  
    </button>  
  );  
}
```

👉 Question: **How would we add types to this?**



Adding Types

```
type DevCampButtonProps = {  
  origin: string;  
  children: React.ReactNode;  
};
```



Applying Types

```
function DevCampButton({ origin, children }: DevCampButtonProps)
```

💥 Type Safety

```
<DevCampButton origin={123}>  
  Click me!  
</DevCampButton>
```

👉 TypeScript catches this early



Introducing State

```
const [count, setCount] = useState(0);
```

- `count` = value => immutable
- `setCount` = update => can be called anything but there are general standards



Rendering

React re-renders when state changes

👉 UI updates automatically

- React runs your component function again and compares the result to what's already on the page, then updates only what changed.
- React is not refreshing the page – it's just updating parts of it.
- UI is a function of state



Todo App

We'll build:

- Input
- Button
- List



First Step



Question: **What state do we need?**

(Hint: input + todos)



State Setup

```
const [input, setInput] = useState("");  
const [todos, setTodos] = useState<Todo[]>([]);
```



Controlled Input



Question: How do we update state when typing?



Controlled Input

```
<input  
  value={input}  
  onChange={(e) => setInput(e.target.value)}  
>
```



Add Todo



Question: What should happen when we click "Add"?



Add Todo

```
setTodos([ ... todos, newTodo]);
```



Render List

```
todos.map(todo => (  
  <li key={todo.id}>{todo.text}</li>  
))
```

Problem

Refresh the page → data disappears



Homework

1. Mark items as complete in the to-list
2. Persist todos using localStorage



Better structure + Pokédex app